This chapter constructs four economic benchmarks that span major application domains where reinforcement learning has been deployed in production. For each benchmark, we review the deployment literature, define a formal MDP, and compare RL against practitioner heuristics using a two-tier evaluation structure: Tier 1 (small complexity parameter) where exact DP is feasible, and Tier 2 (large complexity parameter) where only RL and heuristics run.

0.1 Benchmark 1: Transportation and Logistics

Ride-hailing platforms face a real-time matching problem: given a pool of available drivers and incoming ride requests, assign drivers to passengers to maximize platform efficiency. The problem is naturally sequential because each assignment changes the spatial distribution of available drivers, affecting future match quality.

? formalized DiDi's ride-hailing dispatch as a large-scale multi-agent RL problem. The state space encodes driver locations, passenger request queues, and time-of-day features across a city partitioned into hexagonal zones. Actions are driver-to-request assignments. The reward combines driver income, passenger wait time, and order completion rates. Their deployed system achieved 0.5–2% improvements in key metrics including driver income and order completion rates compared to previous production baselines, and the peer-reviewed evaluation confirmed that the RL dispatcher consistently outperformed combinatorial optimization baselines across multiple Chinese cities.[1] At DiDi's scale of tens of millions of daily rides, these percentage gains translate to substantial aggregate efficiency improvements.

Scaling RL dispatch to city-wide operations introduces a coordination challenge: thousands of drivers must be assigned simultaneously, and each driver's optimal action depends on what other drivers do. **?** addressed this by applying mean-field multi-agent RL, where each driver conditions on the aggregate distribution of other drivers rather than their individual states. This approximation makes the multi-agent problem tractable while preserving the essential spatial interaction structure: a driver's expected pickup time depends on how many other drivers are nearby, which the mean-field distribution captures. Experiments on real DiDi data showed improved fleet utilization compared to single-agent RL baselines.

? reported on RL deployment at Lyft, providing a complementary perspective from a different platform and market. Their system optimizes driver dispatching and positioning jointly, using a value decomposition architecture that assigns credit to individual driver-level decisions within the global objective. The production deployment demonstrated improvements in both

[1] An earlier technical report (**?**) described the system at a high level; **?** provides the formal treatment and controlled experiments.

rider wait times and driver utilization, confirming that the RL dispatch paradigm generalizes across platforms and regulatory environments.

We formalize the dispatch problem as a zone-based MDP that captures the essential spatial allocation structure of these deployments.

Definition 1 (Zone-Based Dispatch MDP)**.** *The zone-based dispatch MDP* $\mathcal{M}_{\text{disp}}(K)$ *is defined as follows.*

1. State space. $\mathcal{S} = \{0, \ldots, D_{\max}\}^K \times \{0, \ldots, Q_{\max}\}^K$, *where* $D_{\max}$ *is the maximum idle drivers per zone,* $Q_{\max}$ *is the maximum demand queue per zone, and* K *is the number of zones. A state* $s = (d_1, \ldots, d_K, q_1, \ldots, q_K)$ *records idle drivers and queued demand per zone. The state space size is* $|\mathcal{S}| = (D_{\max} + 1)^K \cdot (Q_{\max} + 1)^K$.

2. Action space. $\mathcal{A} = \{0, 1, \ldots, K\}$, *with* $|\mathcal{A}| = K + 1$. *Action* 0 *is no rebalancing; action* $k \in \{1, \ldots, K\}$ *rebalances one driver from zone* k *to the zone with the highest demand queue.*

3. Transition function. *Each period: (i) new demand arrives at zone* k *as a Poisson random variable with rate* λ_k, *added to the queue; (ii) matches occur as* $\min(d_k, q_k)$ *per zone; (iii) matched drivers return to the idle pool next period; (iv) any rebalancing action transfers one driver between zones.*

4. Reward function.

$$r(s, a) = f \cdot \sum_{k=1}^{K} \min(d_k, q_k) - c_r \cdot \mathbb{1}\{a > 0\}, \tag{1}$$

   where f *is the fare per match and* c_r *is the rebalancing cost.*

5. Discount factor. $\gamma = 0.95$.

The Bellman optimality equation is

$$V^*(s) = \max_{a \in \mathcal{A}} \left[\mathbb{E}[r(s, a)] + \gamma \sum_{s'} P(s'|s, a)\, V^*(s') \right]. \tag{2}$$

The complexity grows as $(D_{\max} + 1)^K \cdot (Q_{\max} + 1)^K$, which is exponential in K. With $D_{\max} = Q_{\max} = 5$, the state space reaches 6^{2K}: 1,296 states at $K = 2$, rising to over 60 million at $K = 5$. Value Iteration is feasible for $K \leq 3$ and infeasible beyond.

The heuristic set $\mathcal{H}$ contains three policies. The *no-rebalance* policy never moves drivers between zones, relying entirely on organic arrivals to clear demand queues. The *nearest-match* heuristic rebalances one driver from the zone with the most idle drivers to the zone with the highest demand

queue, a greedy spatial matching rule. The *proportional* heuristic rebalances toward zones where the demand-to-driver ratio is highest, approximating a fluid model solution.

We run exact DP for $K \leq 3$ and DQN for all $K \in \{2, 3, 4, 5\}$.[2] Figure **??** plots computation time and policy quality. Table **??** reports numerical results.

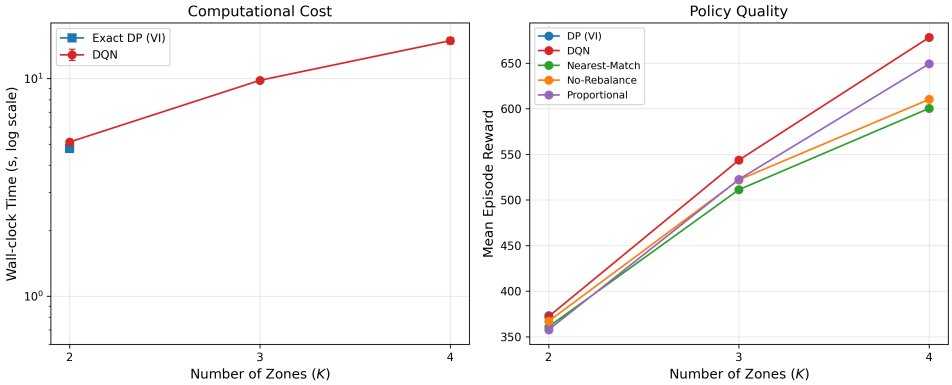

Figure 1: Dispatch benchmark. Left: wall-clock time (log scale) vs. number of zones K. Right: mean episode reward for all methods. DQN learns effective rebalancing policies that outperform greedy heuristics at all tested scales.

K	$\lvert\mathcal{S}\rvert$	DP Reward	DP Time	DQN Reward	Nearest	No-Rebal	Proportional
2	256	373.3	4.77	372.1 ± 1.1	361.0	367.0	357.7
3	4,096	—	—	543.6 ± 1.7	511.2	522.0	522.6
4	65,536	—	—	678.1 ± 3.2	600.4	610.2	649.2

The dispatch benchmark isolates the matching and allocation archetype common across ride-hailing, organ donation, and labor markets. The key insight is that myopic matching ignores the spatial externality of driver positioning; RL policies learn to sacrifice short-run matches for better long-run supply distribution across zones.

0.2 Benchmark 2: Dynamic Pricing and Revenue Management

Dynamic pricing requires setting prices that respond to demand conditions, inventory levels, and competitor behaviour. The sequential structure arises

[2]DQN architecture: two-layer MLP (128-64, ReLU). Training: replay buffer 10,000, batch size 128, Adam at $\alpha = 10^{-3}$, target network sync every 50 episodes, linear ϵ-decay from 1.0 to 0.05 over 60% of episodes. Episode counts: 2,000 for $K \leq 3$; 5,000 for $K = 4$; 10,000 for $K = 5$. Episode horizon: 30 periods. All DQN results averaged over 3 seeds.

because today's price affects both current sales and the inventory available for future periods.

The classical foundation for dynamic pricing is the intensity control framework of **?**, who showed that for perishable inventory with Poisson demand, the optimal price trajectory can be characterized analytically under specific demand models. Their results establish the theoretical benchmark: any RL-based pricing system should match or exceed the Gallego-van Ryzin solution in settings where its assumptions hold, and demonstrate value in the richer settings where those assumptions break down.

Alibaba deployed RL for dynamic pricing on its AliExpress platform. **?** described a learning-based approach to real-time price adjustment, where the state space included current inventory levels, historical sales velocity, and competitor pricing signals, and actions were discrete price adjustments. The system was evaluated on real transaction data from AliExpress and demonstrated improvements in revenue relative to fixed-price and rule-based baselines. The KDD paper focuses on the algorithmic design and controlled evaluation rather than platform-wide revenue attribution.

? demonstrated the value of incorporating microeconomic choice theory into scalable dynamic price experimentation. Their approach combines structural demand estimation with RL-based policy optimization, showing that embedding economic theory into the reward and transition structure improves both sample efficiency and interpretability relative to purely model-free approaches.

? applied RL to hotel revenue management, a domain where the classical approach relies on booking limit controls derived from dynamic programming under parametric demand models. Their formulation treats each hotel property as a finite-horizon MDP: the state encodes current occupancy by room type and remaining days until check-in, actions are accept/reject decisions for incoming reservation requests at posted prices, and the reward is cumulative revenue. The RL agent, trained on historical booking data, learned pricing policies that adapted to demand fluctuations more responsively than static booking-limit heuristics. Results showed that RL-based policies matched or exceeded the performance of well-tuned classical revenue management systems, with particular advantages during demand regime shifts where parametric models required manual recalibration.

We formalize a hotel revenue management MDP that captures the essential pricing-under-scarcity structure of these deployments.

Definition 2 (Hotel Revenue Management MDP)**.** *The hotel revenue management MDP $\mathcal{M}_{\text{hotel}}(C)$ is defined as follows.*

1. State space. $\mathcal{S} = \{0, \ldots, T-1\} \times \{0, \ldots, C\} \times \{0, \ldots, D-1\}$, *where T is the booking horizon, C is the hotel capacity, and D is the number of demand intensity bins. A state (t, I, d) records days until check-in,*

remaining rooms, and recent demand intensity. The state space size is $|\mathcal{S}| = T \cdot (C+1) \cdot D$.

2. Action space. $\mathcal{A} = \{p_1, \ldots, p_M\}$, *a set of* $M = 5$ *discrete price tiers.*

3. Demand model. *Each period, the number of potential arrivals follows* Poisson$(\lambda(d))$, *where* $\lambda(d)$ *depends on the demand intensity bin. Each arrival books with probability*

$$q(p) = \frac{1}{1 + \exp(\alpha \cdot (p - p_{\text{ref}}))}, \tag{3}$$

where $\alpha > 0$ *is the price sensitivity and* p_{ref} *is the reference price.*

4. Reward function. $r(s, a) = p_a \cdot$*bookings, where bookings* $= \min($*arrivals who book*, $I)$.

5. Transitions. *Inventory decrements by bookings. The demand intensity bin transitions stochastically based on realized demand. At* $t = 0$ *(check-in), the episode ends; unsold rooms yield zero revenue.*

6. Discount factor. $\gamma = 1.0$ *(finite horizon, no discounting).*

The Bellman equation for this finite-horizon problem is

$$V_t^*(I, d) = \max_{p \in \mathcal{A}} \left[\mathbb{E}\left[p \cdot B + V_{t-1}^*(I - B, d') \right] \right], \quad V_0^*(I, d) = 0, \tag{4}$$

where B is the random number of bookings and d' is the next demand intensity bin. The state space is $T(C+1)D$, which grows linearly in C. With $T = 30$ and $D = 3$, the state space is $90(C+1)$: approximately 990 states at $C = 10$ and 7,290 at $C = 80$. Although the growth is linear, exact DP requires computing the booking probability distribution for each state-action pair, making it slow for large C.

The heuristic set $\mathcal{H}$ contains three policies. The *fixed price* policy always charges the reference price p_{ref}, ignoring both inventory and time. The *EMSR-b* policy (Expected Marginal Seat Revenue) is a standard airline and hotel heuristic that raises prices when inventory is scarce relative to remaining time.[3] The *myopic* policy selects the price maximizing immediate expected revenue $p \cdot \lambda(d) \cdot q(p)$, ignoring future scarcity.

We run exact DP for $C \leq 20$ and DQN for all $C \in \{10, 20, 30, 50, 80\}$.[4] Figure **??** plots computation time and revenue. Table **??** reports numerical results.

[3] Our EMSR-b implementation uses a simplified rule: if $I/(C \cdot t/T) > 1.2$, price low; if < 0.5, price high; otherwise price at reference. This captures the qualitative structure of capacity-dependent pricing without requiring fare class nesting.

[4] DQN: same architecture as previous benchmarks (128-64 MLP). Episode counts: 2,000 for $C \leq 20$; 5,000 for $C = 30$; 8,000 for $C = 50$; 12,000 for $C = 80$. Episode horizon: $T = 30$ periods.

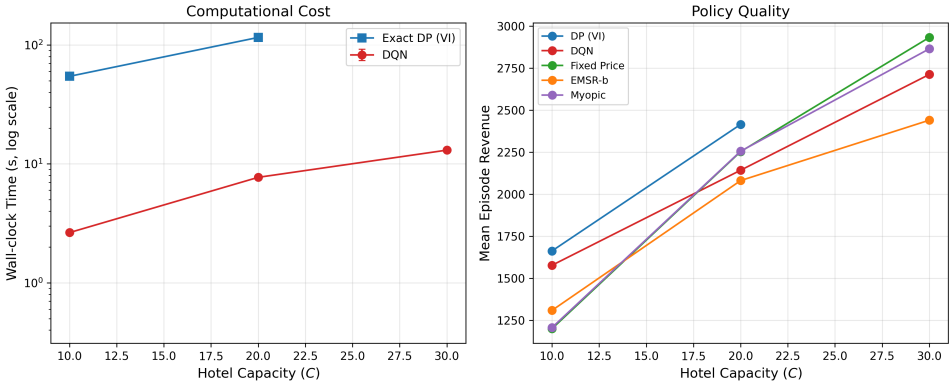

Figure 2: Hotel revenue management benchmark. Left: wall-clock time vs. capacity C. Right: mean episode revenue. DQN learns inventory-dependent pricing that outperforms the fixed-price and myopic baselines, approaching the EMSR-b heuristic and exceeding it at larger capacities.

C	$\lvert\mathcal{S}\rvert$	DP Revenue	DP Time	DQN Revenue	Fixed	EMSR-b	Myopic
10	660	1662.6	54.51	1577.2 ± 20.6	1200.0	1308.8	1206.6
20	1,260	2415.4	115.74	2142.9 ± 70.7	2253.6	2081.2	2256.0
30	1,860	—	—	2712.6 ± 91.2	2932.8	2440.4	2865.0

The hotel benchmark tests the pricing archetype: choosing prices over a finite horizon with perishable inventory. The key economic tension is between selling early at low prices (guaranteed revenue) and waiting for higher-value customers (risky but potentially more profitable). RL policies learn state-dependent pricing schedules that simple heuristics cannot express.

0.3 Benchmark 3: Operations and Energy Management

Operational control problems, where a system must be continuously adjusted to maintain performance under changing conditions, are natural MDPs. The state is the current system configuration and environmental conditions; actions are control adjustments; rewards reflect operational cost or efficiency.

Google applied machine learning to data center cooling, autonomously controlling fans, chillers, and other equipment to maintain safe temperatures while minimizing energy usage. **?** reported that the initial system, based on neural network predictions of Power Usage Effectiveness (PUE), reduced cooling energy consumption by up to 40%, yielding a 15% reduction in overall PUE. **?** subsequently formalized the problem as a model-predictive control task and deployed a safe RL approach that further improved cooling efficiency while providing formal safety guarantees against temperature constraint violations. The state space included over 100 sensor readings (temperatures, pressures, flow rates); actions were setpoints for

cooling equipment. The system operated continuously in production, adapting to seasonal and load-driven variations without manual intervention.

We formalize a multi-zone data center cooling problem as our benchmark MDP.

Definition 3 (Data Center Cooling MDP)**.** *The data center cooling MDP $\mathcal{M}_{\mathrm{dc}}(N)$ is defined as follows.*

1. State space. $\mathcal{S} = \{0, \ldots, T_{\mathrm{bins}}-1\}^N \times \{0, \ldots, L_{\mathrm{bins}}-1\}^N \times \{0, \ldots, E_{\mathrm{bins}}-1\}$, *where T_{bins} is the number of temperature bins per zone, L_{bins} is the number of server load bins per zone, E_{bins} is the number of external temperature bins, and N is the number of zones. The state space size is $|\mathcal{S}| = T_{\mathrm{bins}}^N \cdot L_{\mathrm{bins}}^N \cdot E_{\mathrm{bins}}$.*

2. Action space. $\mathcal{A} = \{c_1, \ldots, c_M\}^N$, *where each c_j is a discrete cooling intensity level. The joint action space size is M^N, exponential in the number of zones.*

3. Thermal dynamics. *For each zone k, the next-period temperature evolves as*

$$T_{k,t+1} = T_{k,t} + \alpha \cdot L_{k,t} - \beta \cdot c_{k,t} + \delta \cdot E_t + \varepsilon_{k,t}, \tag{5}$$

   where α is the heat gain from server load, β is the cooling effectiveness, δ is the external temperature influence, and $\varepsilon_{k,t}$ is i.i.d. noise.

4. Cost function. *The per-period cost is*

$$c(s,a) = \sum_{k=1}^{N} \left[\kappa_1 c_k + \kappa_2 c_k^2 + \Gamma \cdot \mathbb{1}\{T_k \geq \bar{T}\} \cdot (T_k - \bar{T} + 1) + \xi |T_k - T^*| \right], \tag{6}$$

   where κ_1, κ_2 are energy cost coefficients (linear and quadratic), Γ is the overheat penalty, $\bar{T}$ is the overheat threshold, T^* is the target temperature, and ξ is the comfort penalty coefficient.

5. Reward function. $r(s,a) = -c(s,a)$.

6. Transitions. *Temperature evolves per* (**??**) *and is discretized. Server load and external temperature follow independent Markov chains.*

7. Discount factor. $\gamma = 0.95$.

The Bellman optimality equation is

$$V^*(s) = \max_{a \in \mathcal{A}} \left[r(s,a) + \gamma \sum_{s'} P(s'|s,a)\, V^*(s') \right]. \tag{7}$$

The state space grows as $T_{\text{bins}}^N \cdot L_{\text{bins}}^N \cdot E_{\text{bins}}$, which is exponential in N. With $T_{\text{bins}} = 6$, $L_{\text{bins}} = 4$, and $E_{\text{bins}} = 3$, a single zone has $6 \cdot 4 \cdot 3 = 72$ states; two zones have $6^2 \cdot 4^2 \cdot 3 = 1{,}728$ states; four zones exceed 5×10^6 states. The action space also grows exponentially: with 4 cooling levels per zone, the joint action space is 4^N, reaching 256 at $N = 4$. Value Iteration is feasible for $N = 1$ and infeasible beyond.

The heuristic set $\mathcal{H}$ contains three policies. The *fixed setpoint* policy applies medium cooling intensity ($c = 0.5$) to all zones regardless of conditions. The *load-proportional* policy sets cooling intensity proportional to server load at each zone: high load triggers high cooling, idle servers receive minimal cooling. The *PID controller* adjusts cooling based on the temperature error from target, with proportional and integral terms that smooth the response over time.

We run exact DP for $N = 1$ and DQN for all $N \in \{1, 2, 3\}$.[5] Figure **??** plots computation time and operating cost. Table **??** reports numerical results.

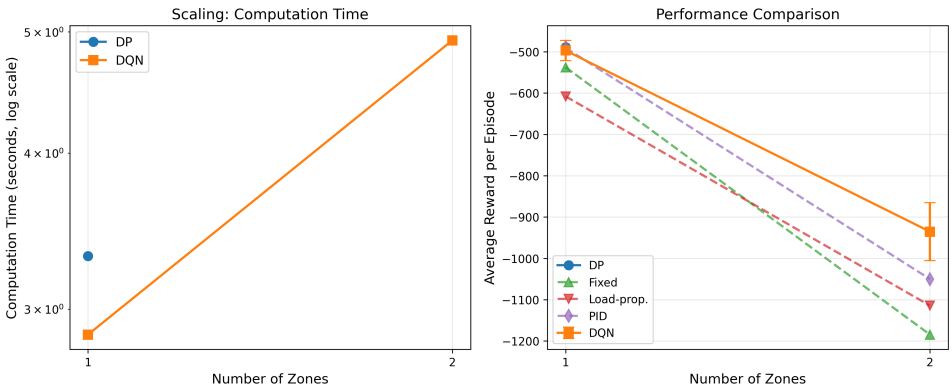

Figure 3: Data center cooling benchmark. Left: wall-clock time vs. number of zones N. Right: mean episode reward (negative operating cost). The fixed setpoint wastes energy on idle zones. Load-proportional cooling improves efficiency but ignores thermal dynamics. RL learns zone-aware cooling schedules that balance energy cost against overheating risk.

Zones	States	DP Time	DP Reward	DQN Time	DQN Reward	Fixed	PID
1	72	3.31	-489.18	2.86	-497.05	-537.58	-490.82
2	1728	—	—	4.92	-935.30	-1184.42	-1050.10

The datacenter benchmark tests the continuous control archetype. The multi-zone structure creates coupling between zones (through external tem-

[5]DQN: same architecture (128-64 MLP). Episode counts: 500 for $N = 1$; 1,000 for $N = 2$; 2,000 for $N = 3$. Episode horizon: 20 periods. All DQN results averaged over 3 seeds. The transition enumeration for exact DP grows as $(3 \cdot T_{\text{bins}})^N \cdot L_{\text{bins}}^N \cdot E_{\text{bins}}$ per state-action pair, making DP impractical for $N \geq 2$.

perature and shared cooling infrastructure) that single-zone heuristics ignore. The PID controller adapts to temperature deviations but does not anticipate load changes; RL policies learn predictive cooling strategies that pre-emptively adjust cooling before temperatures reach critical thresholds.

0.4 Benchmark 4: Supply Chain and Inventory Management

Inventory management is among the oldest sequential decision problems in operations research. A firm must choose how much to order at each period, balancing the cost of holding excess stock against the cost of unmet demand, while facing uncertain future demand. In multi-echelon supply chains, this problem compounds: each stage's order becomes the next stage's demand, creating coupled dynamics across the network.

The classical theory begins with **?**, who established the optimal structure for two-echelon systems under specific cost and demand assumptions. Their echelon base-stock policy, where each stage orders up to a target level defined in terms of total downstream inventory, remains the analytical benchmark. Extensions to general networks, nonstationary demand, and capacity constraints have proven analytically intractable in most cases, motivating computational approaches.

? conducted a systematic evaluation of deep RL for inventory management across a range of supply chain configurations. Their study compared DQN and PPO agents against classical base-stock policies and found mixed results: RL matched or slightly exceeded classical methods in simple settings but struggled with multi-echelon problems where the state space grows combinatorially with the number of stages. The key bottleneck was sample efficiency; training RL agents to convergence required far more demand realizations than are typically available from historical data, and the trained policies were sensitive to distribution shift when demand patterns changed. These findings highlight a recurring theme in RL applications: the gap between algorithmic capability in simulation and reliable deployment under real-world non-stationarity.

? extended this investigation to multi-echelon systems with more complex network topologies, including divergent and convergent supply chains. Using a proximal policy optimization (PPO) agent with a state representation encoding inventory levels and pipeline orders across all echelons, they found that deep RL could discover reorder policies that outperformed myopic heuristics in networks where the echelon base-stock structure does not apply. However, the performance advantage was most pronounced in settings with strong inter-echelon coupling, precisely the cases where classical methods lack closed-form solutions, while in simpler serial chains the base-stock policy remained competitive or superior.

We formalize a serial multi-echelon inventory system as our benchmark MDP.

Definition 4 (Multi-Echelon Inventory MDP)**.** *The serial multi-echelon inventory MDP $\mathcal{M}_{\text{inv}}(K)$ is defined as follows.*

1. State space. $\mathcal{S} = \{0, \ldots, I_{\max}\}^K$, *where $I_{\max}$ is the maximum inventory per echelon and K is the number of echelons. A state $s = (I_1, \ldots, I_K)$ records on-hand inventory at each echelon. The state space size is $|\mathcal{S}| = (I_{\max} + 1)^K$.*

2. Action space. $\mathcal{A} = \{0, 1, \ldots, Q_{\max}\}$, *the order quantity for echelon 1 (the most downstream echelon facing customer demand).* $|\mathcal{A}| = Q_{\max} + 1$.

3. Demand. *Customer demand arrives at echelon 1 as $D \sim \text{Poisson}(\lambda)$.*

4. Dynamics. *Each period: (i) demand D is realized; (ii) echelon 1 fulfills $\min(I_1, D)$ units; (iii) unfilled demand is backlogged; (iv) each upstream echelon $k > 1$ ships $\min(I_k, \textit{deficit}_{k-1})$ downstream; (v) the order of q units arrives at echelon K from an external supplier with infinite capacity (lead time $L = 1$).*

5. Cost function. *The per-period cost is*

$$c(s, a, D) = h \sum_{k=1}^{K} \max(I'_k, 0) + b \cdot \max(0, D - I_1) + c_o \cdot q, \tag{8}$$

   where h is the per-unit holding cost, b is the per-unit backorder cost, c_o is the per-unit ordering cost, and I'_k is the post-shipment inventory.

6. Reward. $r(s, a) = -\mathbb{E}_D[c(s, a, D)]$ *(negative expected cost).*

7. Discount factor. $\gamma = 0.95$.

The Bellman optimality equation is

$$V^*(s) = \max_{q \in \mathcal{A}} \left[\mathbb{E}_D \left[-c(s, q, D) + \gamma V^*(s') \right] \right], \tag{9}$$

where s' is the post-shipment, post-demand inventory vector. The state space $(I_{\max} + 1)^K$ grows exponentially in K: with $I_{\max} = 15$, it is 256 at $K = 2$, 4,096 at $K = 3$, 65,536 at $K = 4$, and over 10^6 at $K = 5$. Value Iteration is feasible for $K \leq 3$ and becomes impractical beyond.

The heuristic set $\mathcal{H}$ contains three policies. The *base-stock* policy orders up to a target level $S^* = F_D^{-1}(b/(b + h))$ whenever inventory at echelon 1 falls below S^*, where F_D is the demand CDF. This is the newsvendor solution applied independently to echelon 1, ignoring upstream coupling. The (s, S) *policy* places an order of $S - I_1$ units whenever $I_1 < s$, with s and S set by rule of thumb. The *fixed order* policy orders $\lceil \lambda \rceil$ units every period regardless of state, a naive replenishment rule.

We run exact DP for $K \leq 3$ and DQN for all $K \in \{2, 3, 4, 5\}$.[6] Figure **??** plots computation time and cost. Table **??** reports numerical results.

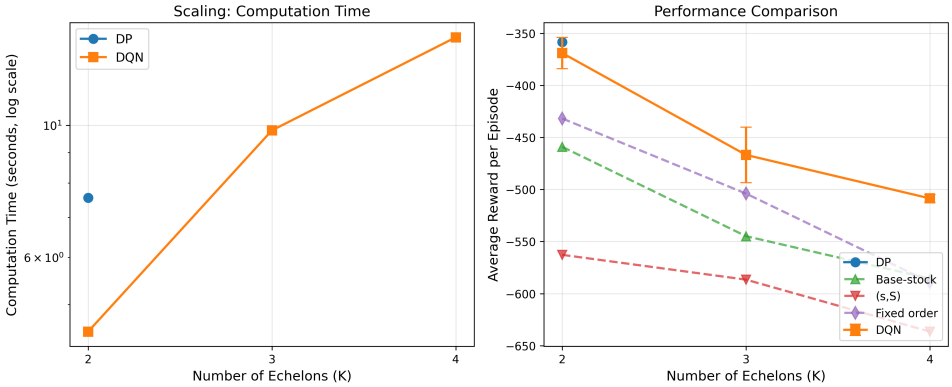

Figure 4: Multi-echelon inventory benchmark. Left: wall-clock time vs. number of echelons K. Right: mean episode reward (negative cost). The base-stock heuristic, optimal for a single echelon, degrades as the supply chain lengthens; RL learns coordinated ordering that accounts for upstream inventory.

K	States	DP Time	DP Reward	DQN Time	DQN Reward	Base-stock	(s,S)
2	81	7.55	-358.18	4.50	-368.87	-458.98	-562.76
3	729	—	—	9.81	-466.74	-544.84	-586.60
4	6561	—	—	14.06	-508.51	-585.52	-636.53

The inventory benchmark tests the quantity choice archetype. The multi-echelon structure introduces coupling between echelons that single-echelon heuristics ignore. The base-stock policy is provably optimal for the single-echelon newsvendor problem, but extending it independently to each echelon in a serial system is suboptimal: it fails to account for how upstream inventory affects downstream replenishment lead times. RL policies learn coordinated ordering strategies that exploit the full state vector.

0.5 Summary of Benchmark Suite

Table **??** summarizes the four benchmarks, their economic archetypes, complexity dials, state space ranges, and the principal lesson each teaches about the relationship between RL and classical methods.

Taken together, the four benchmarks cover major economic decision archetypes: spatial allocation (transport dispatch), dynamic pricing with perishable inventory (hotel), operational control (data center cooling), and

[6] DQN: same architecture (128-64 MLP). Episode counts: 3,000 for $K \leq 3$; 8,000 for $K = 4$; 15,000 for $K = 5$. Episode horizon: 30 periods. State features: $I_k/I_{\max}$ for each echelon.

Table 1: Summary of the four economic benchmarks.

#	Benchmark	Archetype	Dial	$\lvert\mathcal{S}\rvert$ Range	Lesson
1	Transport	Matching	K (zones)	1,296–60×10^6	Spatial externalities
2	Hotel RM	Pricing	C (capacity)	990–7,290	Perishable inventory
3	Datacenter	Continuous control	N (zones)	72–5×10^6	Multi-zone coupling
4	Inventory	Quantity choice	K (echelons)	256–10^6	Supply chain coupling

multi-echelon quantity choice (inventory). Each benchmark confirms the two-tier evaluation structure: RL matches DP where DP is feasible (Tier 1 credibility) and outperforms practitioner heuristics where DP is not (Tier 2 value).

0.6 Common Patterns and Open Challenges

Several patterns emerge across the four benchmarks and the deployed systems they represent. The most successful applications share a common structure: high-frequency decisions, abundant logged data for offline pre-training, and a well-defined scalar reward, with nearly all production systems adding safety mechanisms such as action clipping, fallback to rule-based policies, or conservative offline training before online deployment. The resulting gains are often modest in percentage terms (0.5–2% for dispatch, 15% for trade execution) but economically large due to the scale of operations. Significant challenges remain: reward misalignment can produce policies that optimize a metric while degrading the economic outcome; multi-objective settings require reward engineering that current practice handles through ad hoc weighted sums; sample efficiency is a bottleneck where experimentation is costly or risky; and interpretability of RL policies lags behind traditional approaches, limiting adoption in regulated industries.